AMATH 345 Final Project: 3D Space Reconstruction from 2D Images

Ariq Ishfar, Gaurika Gupta, & Caeden Harmer

December 1, 2025

Contents

1 Topic Overview & Motivation **4**
1.1 Introduction to 3D Reconstruction . . . 4
1.2 Motivation . . . 4
1.3 Project Objectives . . . 5

2 Background Theory **5**
2.1 The Pinhole Camera . . . 5
2.2 Camera Parameter Decomposition . . . 6
2.3 Depth Perception . . . 8

3 Literature Review **9**
3.1 Building Rome with Convex Optimization: SfM, SLAM, and Batch Adjustment . . . 9
3.2 Multi-View Stereo Revisited: Historical Iterations of MVS Compared with Modern Techniques . . . 10
3.3 PLAN 3D: Viewpoint and Trajectory Optimization for Aerial Multi-View Stereo Reconstruction . . . 11
3.4 NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis . . . 11

4 Camera Parameter Estimation **12**
4.1 The Pinhole Camera Model . . . 12
4.2 Intrinsic Parameters Matrix (K) . . . 12
4.3 Extrinsic Parameters Matrix . . . 13
4.4 Estimating Camera Parameters . . . 13

5 Feature Matching and Detection **13**
5.1 SIFT . . . 13
5.1.1 Extrema Detection . . . 13
5.1.2 Keypoint Localization . . . 14
5.1.3 Orientation Assignment . . . 14
5.1.4 Keypoint Descriptor Creation . . . 14
5.1.5 Keypoint Matching . . . 15
5.2 ORB (Oriented FAST and Rotated BRIEF) . . . 15
5.2.1 The oFAST Keypoint Detector . . . 15
5.2.2 The rBRIEF Descriptor . . . 15
5.2.3 Performance . . . 16
5.3 Local Feature Transformer (LoFTR) . . . 16
5.3.1 Local Feature Extraction . . . 17
5.3.2 Local Feature Transformer (LoFTR) Module . . . 17
5.3.3 Matching Module . . . 18
5.3.4 Coarse-to-Fine Module . . . 18
5.3.5 Performance . . . 18

6 Depth Estimation and Triangulation **20**
6.1 Fundamental and Essential Matrices . . . 20
6.2 Relative Rotation and Translation Matrix . . . 20
6.3 Linear Triangulation . . . 21

7 Dense Reconstruction **21**
7.1 Dense Reconstruction Pipeline . . . 21
7.2 Primary MVS Method Families . . . 22
7.2.1 PatchMatch Stereo . . . 22
7.2.2 Plane Sweeping . . . 22
7.2.3 Learning-Based MVS . . . 22

8 Surface Reconstruction, Meshing, and Visualization from Point Clouds **22**
8.1 Poisson Surface Reconstruction . . . 23
8.2 Surface Reconstruction Algorithms . . . 23
8.2.1 Poisson Surface Reconstruction . . . 23
8.3 Texturing the Mesh . . . 23
8.3.1 Per-Vertex Color Mapping . . . 24
8.3.2 UV Texture Mapping . . . 24
8.4 Visualization Tools . . . 24
8.4.1 COLMAP . . . 24
8.4.2 Meshroom (AliceVision) . . . 24
8.4.3 MeshLab . . . 24
8.4.4 Open3D . . . 24

9 Code Demonstration **25**
9.1 Camera Parameter Estimation . . . 25
9.2 Feature Detection and Matching . . . 26
9.3 Setting up the Coordinate Frame . . . 26
9.4 Structure From Motion . . . 27
9.4.1 Find Next Best Image . . . 27
9.4.2 Pose estimation . . . 28
9.4.3 Point Triangulation . . . 28
9.5 Output . . . 29

10 Conclusion and Findings **30**

1 Topic Overview & Motivation

1.1 Introduction to 3D Reconstruction

Three-dimensional (3D) reconstruction is a core challenge in the domains of computer vision, photogrammetry, and applied mathematics. It pertains to the method of deducing the three-dimensional shape and look of a physical object or scene based on a collection of two-dimensional (2D) images. The main challenge is that a singular 2D image is a representation of the 3D world, leading to an unavoidable loss of depth information. Algorithms for 3D reconstruction aim to retrieve this missing information by examining the connections—like parallax and perspective—among several overlapping images captured from various angles [1].

The procedure generally adheres to an organized workflow. Initially, camera calibration is conducted to establish the intrinsic parameters (such as focal length and lens distortion) and extrinsic parameters (location and orientation) for each camera. Subsequently, feature detection and matching locate matching points throughout the image collection. These correspondences enable the assessment of 3D structure using methods such as triangulation and bundle adjustment, leading to a dense point cloud or mesh model that depicts the scene's geometry [2].

1.2 Motivation

The capability to create precise 3D models from ordinary 2D images has significant impacts across various industries and scientific fields, fostering ongoing research and advancement in this field.

Cultural Heritage and Archaeology: 3D reconstruction offers a non-destructive approach to digitally safeguard historical sites, artifacts, and monuments. These digital collections can aid in restoration activities, virtual tourism, and academic research, guaranteeing that cultural heritage remains accessible to future generations, even if the original is harmed or missing. Initiatives such as the digital reconstruction of the ancient city of Palmyra showcase its significant importance (UNESCO, 2020).

Autonomous Systems and Robotics: Self-navigating vehicles, drones, and robotic technologies depend on an accurate comprehension of their three-dimensional surroundings to move and engage safely. 3D reconstruction using onboard cameras is a crucial technology for simultaneous localization and mapping (SLAM), allowing these systems to detect obstacles, plan routes, and function independently in real-time (Cadena et al., 2016).

Medical Imaging and Healthcare: In healthcare, methods based on 3D reconstruction are employed to form models of anatomical structures from slices obtained through MRI or CT scans. These models aid in surgical planning, personalized prosthesis creation, and improving diagnostic precision, resulting in better patient results.

Entertainment, Gaming, and Virtual Reality (VR): The movie and video game sectors utilize photogrammetry—a technique for 3D reconstruction from images—to produce extremely lifelike assets and surroundings. This technology enables the smooth incorporation of physical objects into virtual environments, driving the expansion of immersive VR and augmented reality (AR) experiences.

Topographic Mapping and Geoscience: Using satellite and aerial images, 3D reconstruction methods create digital elevation models (DEMs) and topographical maps. These are essential for city development, crisis management, ecological observation, and geological examination.

1.3 Project Objectives

This report attempts to offer a clear and organized examination of the fundamental algorithms that support the 3D reconstruction pipeline, given the intricacy and dynamic nature of this field of study. We will analyze and assess the mathematical underpinnings of the following crucial phases: surface creation, sparse and dense reconstruction, camera pose estimation, and feature identification and matching. The information acquired from this study will be combined to provide an explanation of a real-world 3D reconstruction, linking abstract ideas to a concrete result.

2 Background Theory

2.1 The Pinhole Camera

A simple model that provides a mathematical description of how three-dimensional scenes are projected onto a camera image is the *pinhole camera* [3]. All information in this section has been adapted from the work of Faisal Qureshi in the previous citation.

The opening of a pinhole camera is described as a point, through which the incoming rays of the object pass through and project an image onto the plane of the camera. Consider a pinhole camera where the camera plane is at distance f from the opening (referred to as *focal length*), and an object of height y placed at distance d from the pinhole, as shown below.

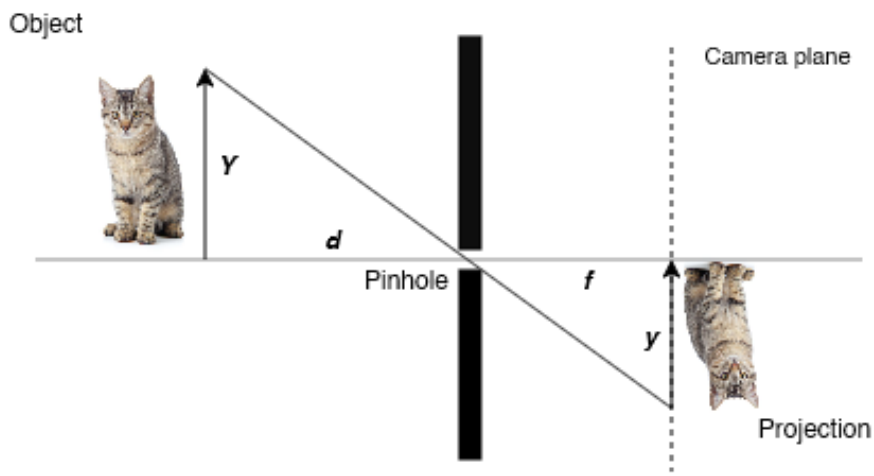


Figure 1: A 2-dimensional analogue of image projection

To find the height of the projection, we can use the similar triangle rule to obtain:

$$\frac{Y}{d} = \frac{y}{f} \Rightarrow y = f\frac{Y}{d}$$

Hence, with the origin being the pinhole, a point (X, Y, Z) in the plane of the object with $Z > 0$ will be projected to:

$$(X, Y, Z) \mapsto \left(f\frac{X}{Z}, f\frac{Y}{Z}\right)$$

in the camera plane. This is a nonlinear mapping and can be inconvenient to work with when we are dealing with transformations. To remedy this, we can use homogeneous coordinates of the form $(X, Y, Z) \mapsto (X, Y, Z, 1)$.

Firstly, note that projections follow *scale invariance*, with:

$$(\lambda X, \lambda Y, \lambda Z) \mapsto \left(f\frac{X}{Z}, f\frac{Y}{Z} \right)$$

and so the set of points $(\lambda X, \lambda Y, \lambda Z)$ for $\lambda > 0$ are mapped to the same point on the camera plane. We can then present the projection transformation as:

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \mapsto \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix}$$

$$= \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$= P\mathbf{X}$$

with the final image coordinates being obtained by dividing the homogenous coordinates by Z.

Suppose that the center of the pinhole projection on the camera plane is different from that of the center of the camera plane, as shown below.

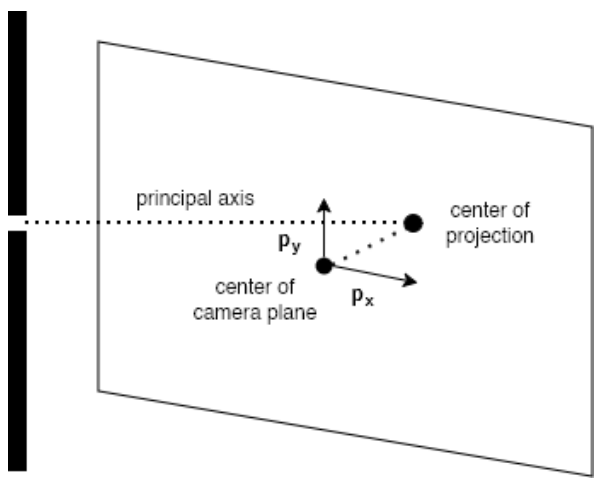


Figure 2: Illustration of offset projection

We can account for this in the camera projection matrix as such:

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \mapsto \begin{bmatrix} fX + p_X Z \\ fY + p_Y Z \\ Z \end{bmatrix} = \begin{bmatrix} f & 0 & p_X & 0 \\ 0 & f & p_Y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

2.2 Camera Parameter Decomposition

The construction of the projection matrix in the previous section involves two coordinate systems, one based on the position of the object with respect to the camera and the other based on the projective camera plane. A reconstruction of a 3D scene from 2D images also needs another system based on

where the camera and the object is in relation to the world.

The parameters derived in the previous section can be described as *intrinsic* parameters, since they are specific to the camera and do not change with the scene being viewed. *Extrinsic* parameters help us bridge the camera scene to the world scene by describing the relationship between them as a product of linear transformations, as shown below [4].

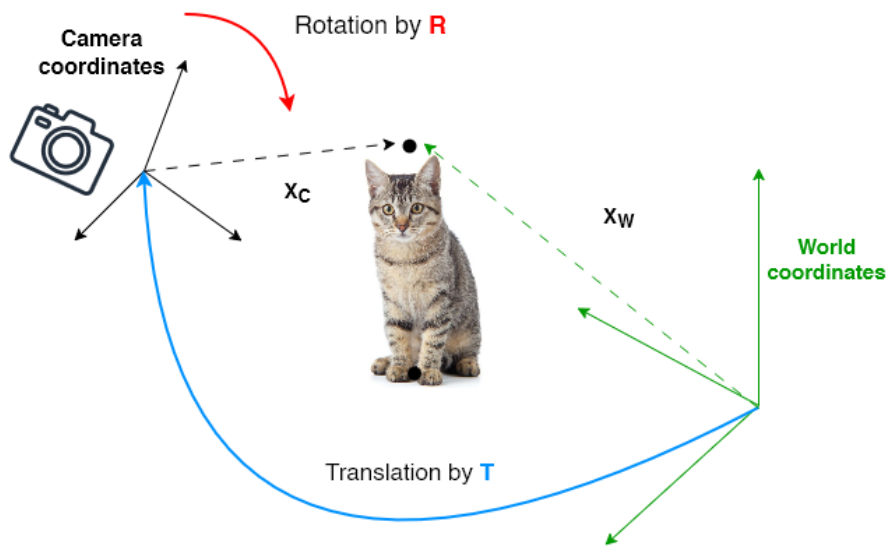


Figure 3: Transformations between world and camera coordianates

Consider a point represented by the coordinates $\mathbf{X}_C$ with respect to the camera and $\mathbf{X}_W$ with respect to the world. We have that the world coordinate system can be transformed to the camera coordinate system by means of a translation T followed by a rotation R (shown above), which are represented as:

$$T = \begin{bmatrix} T_X \\ T_Y \\ T_Z \end{bmatrix} \quad R = \begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix}$$

Using the matrices (with non-homogenous coordinates), we get:

$$\mathbf{X}_C = R[\mathbf{X}_W - T]$$

To apply this to our homogenous coordinates, we have:

$$M_R = \begin{bmatrix} r_1 & r_2 & r_3 & 0 \\ r_4 & r_5 & r_6 & 0 \\ r_7 & r_8 & r_9 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad M_T = \begin{bmatrix} 1 & 0 & 0 & -T_X \\ 0 & 1 & 0 & -T_Y \\ 0 & 0 & 1 & -T_Z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_R M_T = \begin{bmatrix} r_1 & r_2 & r_3 & -(r_1T_X + r_2T_Y + r_3T_Z) \\ r_4 & r_5 & r_6 & -(r_4T_X + r_5T_Y + r_6T_Z) \\ r_7 & r_8 & r_9 & -(r_7T_X + r_8T_Y + r_9T_Z) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

or in block-matrix format:

$$M_R M_T = \begin{bmatrix} R & -RT \\ 0^T & 1 \end{bmatrix}$$

Lastly, we represent the transformation from the world coordinates $\mathbf{X}_W$ to image coordinates $\mathbf{X}_I$ as:

$$\mathbf{X}_I = P M_R M_T \mathbf{X}_W$$

$$\begin{bmatrix} X_I \\ Y_I \\ Z_I \end{bmatrix} = \begin{bmatrix} f & 0 & p_X & 0 \\ 0 & f & p_Y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R & -RT \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$$

with the final image coordinate obtained as $\left(\frac{X_I}{Z_I}, \frac{Y_I}{Z_I}\right)$.

2.3 Depth Perception

Recall from section 1.1 the property of *scale invariance* for any $\lambda > 0$,

$$(\lambda X, \lambda Y, \lambda Z) \mapsto \left(f\frac{X}{Z}, f\frac{Y}{Z}\right)$$

i.e. all points on a line from the pinhole get mapped to the same point on the image plane. This presents the difficult problem of *depth perception* - recovering depth information lost from 3D to 2D projection.

Depth perception methods rely on information outside of image data to provide estimations or heuristics for depth, some of which include:

- With multiple cameras, the spatial information of each camera.
- With images over time, the temporal information of each image.
- Photometric information related to shading, shadows, and reflections as such.
- Textural information of the object such as albedo, light scattering, and differences in how textures look at varying distances.
- Optical properties when using camera lenses, one example being differences in blurring and abberations with distance from the object.

  Note that camera lenses use glass elements to focus light on a single point, whereas a pinhole camera uses a small opening instead.
- Occlusion caused by objects behind in front or behind of other objects.
- Prior and learned information. These include intuitions as the sky being (practically) infinitely far away, buildings being vertical, and relative size and shapes of objects.
- Mechanics of sensing, an example being the time taken by the sensing material (photons in most cases) to be sent out and return to the sensor.

We will evaluate the effectiveness and viability of various different depth perception methods in later sections.

3 Literature Review

Though 3D reconstruction relies heavily upon feature extraction and matching algorithms, there are two steps that are specific and intrinsic to 3D reconstruction:

- **Camera Location Estimation and Sparse Reconstruction**:

  This is usually computed with a Structure-from-Motion (SfM) algorithm, though other techniques such as Simultaneous Localization and Mapping (SLAM) may also be used.

- **Dense Reconstruction**:

  Once the camera locations have been estimated and a sparse reconstruction has been completed, a dense reconstruction algorithm completes the 3D reconstruction with greater resolution. This is often done with the use of a Multi-View Stereo (MVS) algorithm. However, Neural Radiance Field (NeRF) algorithms are gaining traction — particularly for photorealistic and novel view rendering.

The following reviews of literature on this topic will highlight the past, present, and future directions for these two important steps.

3.1 Building Rome with Convex Optimization: SfM, SLAM, and Batch Adjustment

In their paper "Building Rome with Convex Optimization" [5], Han and Yang highlight that for both SfM and SLAM pipelines, a procedure of reconstructing camera poses and 3D landmarks — called *batch adjustment* — is essential.

Batch adjustment usually takes place after the feature detection and mapping steps in a SfM or SLAM pipeline and often assumes the availability of what Han and Yang call a "view graph". These view graphs consist of 3D points $p_k \in \mathbb{R}^3$, camera poses $(R_i, t_i) \in \mathrm{SE}(3)$, and measurements $u_{ik} \in \mathbb{R}^2$ which represent the projection of point p_k onto camera pose (R_i, t_i). From this, an optimization problem of the form

$$\min_{\substack{p_k \in \mathbb{R}^3 \\ (R_i, t_i) \in \mathrm{SE}(3)}} \sum_{(i,k) \in \mathcal{E}} ||u_{ik} - \pi(R_i p_k + t_i)||^2 \tag{1}$$

where $\mathcal{E}$ is the set of all possible connections between points and poses/images and $\pi(v) := [v1; v2]/v3$. In this case, $R_i p_k + t_i$ transforms the point p_k to the image i, which is then projected to 2D via $\pi(v)$ and compared with u_{ik}.

However, as the authors note, this approach has two major issues. Firstly, the optimization problem can only be solved up to scale. In other words, since

$$\pi(R_i p_k + t_i) \equiv \pi(\alpha R_i p_k + \alpha t_i)$$

the objective of the problem is invariant under scaling by a factor of $\alpha > 0$. Secondly, the problem can be difficult to optimize. The problem is highly non-convex and can have an extremely large scale — values for k and i can sometimes range in the tens of thousands. Thus, popular algorithms such as CERES and GTSAM require good initializations and are susceptible to local minima entrapment. COLMAP, a popular SfM pipeline employs an incremental strategy to view reconstruction to ensure stable initialization. However, this is extremely resource intensive and can slow the pipeline for large datasets.

From this, Han and Yang formulate the driving question of their research: Is there an algorithm that solves the bundle adjustment problem without initialization and at scale?

This question prompts their design of a *scaled batch adjustment formula*, which uses learned depth to simplify (1). By using monocular depth prediction models, for each keypoint u_{ik}, one can find a corresponding depth estimate d_{ik}. From this, once can project the 2D keypoint to 3D according to

$$\tilde{u}_{ik} = d_{ik} \begin{bmatrix} u_{ik} \\ 1 \end{bmatrix} \tag{2}$$

This leads to the scaled bundle adjustment (SBA) formula

$$\min_{\substack{p_k \in \mathbb{R}^3 \\ s_i > 0 \\ (R_i, t_i) \in \mathrm{SE}(3)}} \sum_{(i,k) \in \mathcal{E}} w_{ik} ||R_i(s_i \tilde{u}_{ik}) + t_i - p_k||^2 \tag{3}$$

where s_i is a scaling factor that enables correction from previous iterations and w_{ik} are confidence values taken from the depth predictor. This clearly nullifies the issue of scale due to the $\pi(v)$ function and, though it does not completely remove nonconvexity from the problem, what nonconvexity remains is benign.

As the authors highlight, the "XM" solver created using this formula meets most of the criteria initially set out: it is scalable and initialization-free, and any nonconvexity can be shown to be benign. However, it struggles with noise, outliers, and sparsity, as highlighted by its general incompatibility with SLAM pipelines. Furthermore, this solver requires heavily upon the monocular depth prediction model used, which can often be hard to find for data scarce domains such as medicine and space science.

3.2 Multi-View Stereo Revisited: Historical Iterations of MVS Compared with Modern Techniques

As Goesele et al. highlight in their paper "Multi-View Stereo Revisited" [6], modern multi-view stereo algorithms are a far cry from the original window-based algorithms [1]. These algorithms, though they are far more powerful and produce far better quality in their results, are also significantly more complex — making understanding and using them more difficult.

Though there are many forms of these modern algorithms, they can generally be broken down into three primary types:

- **Graph Cuts:** First introduced by Greig, Porteous and Seheult in their 1988 paper "Exact Maxiumum A Posteriori Estimation for Binary Images" [7], this algorithm uses the concept of source and sink nodes (and costed edges between them) to build a graph of an image. This graph can then be "solved" by finding the cut with that separated the image into sources and sinks while maintaining as many high-cost edges as possible.

- **Level Set Evolution:** These algorithms represent and segment images by implicitly defining a shape and evolving it over time to snap to object in the image [8]. For example, one can represent a curve by the equations

  $$\phi(x, y) = 0 \text{ (the boundary)}$$
  $$\phi(x, y) < 0 \text{ (inside)}$$
  $$\phi(x, y) > 0 \text{ (outside)}$$

  This implicit representation has several benefits compared to an explicit representation when it comes to deformation and spatial optimization. It better allows for the splitting of the shapes into multiple pieces, merging multiple pieces together, and other topological changes.

[1] These algorithms simply compare patches of images based on a similarity measure and pick the corresponding depth with the best match.

- **Mesh Evolution:** These algorithms are similar to level set evolutions, but use an explicit representation as opposed to implicit. That is, one can represent a surface or shape as a set of vertices ($v_i = (x_i, y_i, z_i)$) and the edges/faces formed by these vertices [9]. For multi-view stereo applications, the mesh is then deformed by taking the gradient descent on the formulated energy function of general form

  $$E = E_{\text{photo}} + \lambda E_{\text{smooth}}$$

  Though these explicit methods make changing the topology of the region more difficult, mesh evolution algorithms are often better for high-resolution geometries and are generally more efficient.

However, in this paper Goesele et al. argue that simple modifications to the original window-based can produce results with similar accuracy to more complex modern methods. As they explain, their algorithm first constructs a depth map for all input views, using a window-matching algorithm with minimal neighbouring views that returns only strong matches. The depth maps are then merged using a "standard volume" merging method. Though this algorithm can output high-resolution and accurate reconstructions where detailed information is available, it can create holes as the first step only reconstructs for regions with high confidence.

3.3 PLAN 3D: Viewpoint and Trajectory Optimization for Aerial Multi-View Stereo Reconstruction

As previously discussed, any Multi-View Stereo algorithm requires detailed information to give accurate reconstructions. However, blind spots and areas with limited information may be hard to determine before a reconstruction is attempted. As Hepp et al. [10] explain "final reconstruction quality depends to a large degree on the availability and quality of the set of input images." Though for a highly specific purpose (aerial building-scale reconstructions with quadrotors/aerial robots), Hepp et al. propose an semi-automated process for capturing scenes to a high degree of detail:

1. The region/area to be modelled and a pattern of initial viewpoints are defined
2. The initial viewpoints are captured and the images are processed for an initial reconstruction; an occupancy map which locates dense and unknown regions is also computed
3. A secondary viewpoint path is created to capture the unknown region, this path is only allowed to travel through known free space, making it possible to be run autonomously
4. These secondary viewpoints are processed and a final high-resolution reconstruction is created

Overall, this paper demonstrates how technical deficiencies can sometimes be more easily solved by improving the quality of the inputs through a standardized and semi-automated process.

3.4 NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis

In this paper, Mildenhall et al. [11] offer a modern, use-specific alternative to Multi-View Stereo algorithms in the form of Neural Radiance Fields (NeRFs). This method represents scenes as 5D functions, comprised of a viewing direction (θ, ϕ) and explicit positions in space (x, y, z). A density value for each position is also included which, as the authors note, can be seen as a differential opacity that measures the "radiance" accumulated by passing through the point/position.

Then, to render a NeRF from a particular viewpoint:

1. Send "camera rays" through the scene as represented by the input images to generate a set of 3D points

2. Use these points and the associated viewing directions to estimate colours and densities with a neural network
3. Use traditional rendering techniques to process those colours and densities to create a novel 2D image

As may be clear by the last step in its implementation, NeRF is best suited for photorealistic novel view creation, rather than reconstructing accurate 3D geometries.

4 Camera Parameter Estimation

Camera parameter estimation is the foundation of any 3D reconstruction pipeline because it establishes the mathematical link between 3D points in the environment and their corresponding 2D projections in images. This process involves estimating two sets of parameters:

- **Intrinsic parameters**: characterize the camera's internal geometric and optical properties.
- **Extrinsic parameters**: describe the camera's position and orientation in the world.

4.1 The Pinhole Camera Model

The pinhole camera model is the most typical way to describe this imaging process. This model uses a central projection to project a 3D point (X, Y, Z) to a 2D image point (u, v). This projection can be written using homogeneous coordinates as:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = P \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix},$$

where λ is a scale factor representing the projective depth.

The 3×4 projection matrix P can be decomposed as:

$$P = K\,[\,R \mid t\,],$$

where K is the 3×3 camera intrinsic matrix, and $[R \mid t\,]$ is the 3×4 camera extrinsic matrix [2].

4.2 Intrinsic Parameters Matrix (K)

The intrinsic matrix K typically takes the form:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

Focal Length (f_x, f_y): These pixels represent the distance between the image plane and the camera center. They are responsible for the projection's scaling effect.

Principal Point (c_x, c_y): The image center coordinates are those at which the optical axis intersects the image plane. It is usually close to the image's center.

Skew Coefficient (s): This parameter accounts for pixels that are not rectangular. In most current digital cameras, it is almost zero and may often be ignored. [1].

4.3 Extrinsic Parameters Matrix

The extrinsic parameters specify the rigid-body transformation that converts points from the world coordinate system to the camera coordinate system.

Rotation Matrix (R) is a 3×3 orthonormal matrix that represents the camera's orientation.

The Translation Vector (t) is a 3×1 vector that represents the camera's center location in the world coordinate system.

The transformation is defined by

$$X_{\text{camera}} = RX_{\text{world}} + t.$$

4.4 Estimating Camera Parameters

Structure-from-Motion (SfM) methods can recover camera parameters directly from an unordered image set in the absence of a physical calibration target. This method's central idea is to use matched feature points to establish geometric constraints between images.

The Fundamental Matrix (F) is first calculated using the corresponding points between two images. The Fundamental Matrix is transformed into the Essential Matrix (E) if the intrinsic parameters (K) of the camera are known or can be roughly estimated.

The relative rotation R and translation t between the two camera views are contained in [12]:

$$E = [t]_{\times} R,$$

From matched features, E can be estimated using the Eight-Point Algorithm. The translation vector t and the rotation matrix R are then extracted from this matrix using Singular Value Decomposition (SVD). The translation is only recovered up to an unknown global scale factor, which means the initial reconstruction is accurate in shape but not in absolute size. This is a fundamental ambiguity in this process.

A key component of modern SfM systems is this method for determining relative camera pose, which allows programs like COLMAP and VisualSFM to initialize complete 3D models from image collections [13].

5 Feature Matching and Detection

5.1 SIFT

Developed in 2004 by David Lowe at the University of British Columbia, Scale Invariant Feature Transform (SIFT) is a comprehensive keypoint extraction and descriptor computation algorithm [14]. It functions by creating a small region around feature points and using regional coordinate frames to compute a position, local scale, and orientation of each feature [15]. There are four primary steps that define the SIFT algorithm that process keypoint detection, localization, and description:

5.1.1 Extrema Detection

To be scale invariant, the SIFT algorithm needs to be able to detect keypoints at a variety of scales. For example, a curve that is clearly visible at one scale may appear flat when the scale is decreased or the curve is zoomed in on. This has the potential to make keypoint detection difficult at various scales, depending on the image and features.

To rectify this, SIFT uses the process of scale-space filtering for keypoint detection. This introduces the scaling parameter σ, which is used to detect "blobs" of various sizes due to changes in σ. This detection method is called the Laplacian of Gaussian (LoG) and is used to find local maxima across the space defined by (x, y, σ), essentially locating potential keypoints (x, y) at scale σ.

While LoG is effective at finding potential keypoints, it can be computationally heavy and costly. Thus, the Difference of Gaussians is used, which acts as an approximation and is calculated by the difference of Gaussian blurring between images of different scales.

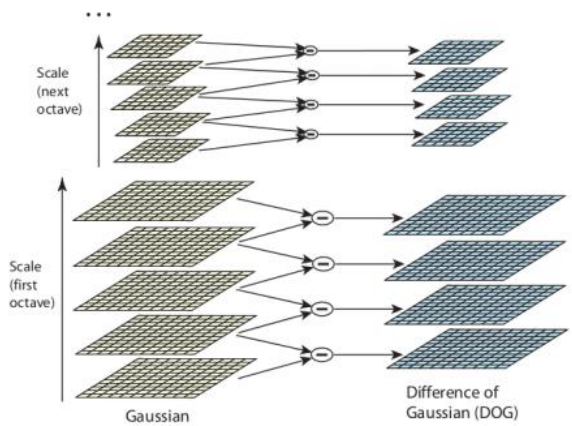


Figure 4: Process of Obtaining the Difference of Gaussian

Subsequently, using the Difference of Gaussian the images are searched for local extrema over different scales.

5.1.2 Keypoint Localization

After potential keypoints are identified, SIFT uses a taylor series expansion of scale space to more accurately locate the extrema, rejecting points for which the intensity falls below a predetermined threshold value. Just as low-contrast points must be removed, the Difference of Gaussian is particularly sensitive to edge detection and, therefore, these edges must be identified and rejected. For this, SIFT utilizes a ratio of the eigenvalues of the Hessian matrix compared to a predetermined upper edge threshold. If this ratio exceeds the threshold, the point is rejected as a keypoint.

5.1.3 Orientation Assignment

To ensure invariance to image rotation and stability of matching, the keypoints must also be assigned an orientation. In SIFT, this is calculated by taking the gradient magnitude and direction in that region, then taking the highest peak in a gradient magnitude weighted histogram for potential orientations. In addition to the highest peak, orientations which have a peak over 80% are also considered in this calculation.

5.1.4 Keypoint Descriptor Creation

Once the aforementioned steps are completed and calculations for each keypoint's position, scale, and orientations are known, a keypoint descriptor can be created. This descriptor is represented by a 128-dimensional vector, based on 8-bin orientation histograms of 16 sub-neighbourhoods of a 16 by 16 neighbourhood around the keypoint [15].

5.1.5 Keypoint Matching

Given the images' keypoint descriptors, keypoints between images are matched by identifying their nearest neighbours in the other. This can be complicated by multiple close matches due to noise or other data inconsistencies. However, in such cases SIFT computes the ratio of the distance to the nearest neighbour versus the second-nearest, discarding the match if it is greater than 80%.

5.2 ORB (Oriented FAST and Rotated BRIEF)

ORB combines the FAST (Features from Accelerated Segment Test) keypoint detector and the BRIEF (Binary Robust Independent Elementary Features) descriptor, incorporating enhancements to boost performance. Its goal is to offer a quick and effective substitute for SIFT.

5.2.1 The oFAST Keypoint Detector

A 16-pixel circle encircling a potential point is examined by the conventional FAST algorithm. A point is categorized as a corner if a contiguous set of N pixels (usually 12) are all either brighter or darker than the center pixel (by a threshold).

Standard FAST is incredibly quick, but it has two main drawbacks:

- It is difficult to choose the "best" N keypoints because it does not provide a measure of corner strength.
- It is neither orientation-aware nor scale-invariant.

oFAST was created to overcome these drawbacks.

To achieve scale invariance, ORB builds a multi-scale image pyramid and identifies FAST keypoints at every level. This enables the system to capture characteristics at various resolutions.

Because FAST by itself does not prioritize keypoints, ORB employs the Harris corner response to evaluate and choose the most stable and unique points. Only the highest-ranked corners are kept, enhancing reliability during the matching process.

For rotation invariance, ORB calculates an orientation for every keypoint employing an intensity centroid approach. It calculates the centroid of pixel intensity values in a patch around the keypoint and establishes the orientation as the angle between the keypoint's position and this centroid. This streamlined method offers a computationally efficient and effective estimation of orientation [16]

5.2.2 The rBRIEF Descriptor

ORB employs a modified version of the BRIEF descriptor after identifying oriented keypoints. The bit-string descriptor BRIEF can be computed and compared relatively quickly. In a smoothed image patch, it simply compares the intensities of a set of predetermined pixel pairs.

Nonetheless, standard BRIEF has a major flaw: it is extremely sensitive to in-plane rotation. ORB addresses this issue by developing "rBRIEF" (rotation-invariant BRIEF):

- **Steering the Descriptor:** ORB steers the predetermined BRIEF sampling pattern using the orientation angle calculated by oFAST. Prior to the intensity comparisons, each keypoint's sampling pattern is rotated to match its orientation.
- **Finding the Best Sampling Pattern:** The authors pointed out that not every pair of pixels in the BRIEF pattern has the same level of discrimination. The 256 most uncorrelated and high-variance pairs were chosen after they examined the variance and correlation of possible sampling

pairs across a sizable training set. By lowering noise and enhancing matching performance, this learned sampling pattern produces a more reliable and discriminative descriptor [17].

5.2.3 Performance

The pairing of oFAST and rBRIEF provides ORB its unique characteristics:

- **Speed:** ORB outpaces SIFT by a considerable margin and is much quicker than SURF, as it utilizes straightforward intensity comparisons to generate a binary descriptor.
- **Efficiency:** Binary descriptors can be matched quickly using the Hamming distance, achieved through a bitwise XOR operation followed by a count of bits, a fundamental task that contemporary CPUs handle with great efficiency [18].
- **Robustness:** It shows strong resistance to image noise and changes in viewpoint, exceeding SIFT in certain rotational and viewpoint evaluations, although it may be less resilient to major illumination variations compared to SIFT [16].
- **License-Free:** In contrast to SIFT and SURF, which hold patents, ORB is available for use without charge for commercial and academic applications, leading to its broad acceptance.

5.3 Local Feature Transformer (LoFTR)

With the emergence of *deep learning* from the 2010's onward, numerous feature matching algorithms have been introduced that take advantage of improvements in computational power and deep learning methods. A notable one being *Local Feature Transformer* or (LoFTR), developed by Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou [19]. Unless stated otherwise, all information in this section has been taken from the authors' work.

Most feature matching algorithms, including SIFT and ORB, involve a *feature detection* phase, where points of interest identified in the image are used to map points in one image to the other. However, many factors can result in the algorithm being unable to identify features such as variable lighting, motion blur, repetitive or plain textures, and viewpoint changes.

LoFTR forgoes feature detection in favor of directly producing matching points between image pairs. The algorithms consists of:

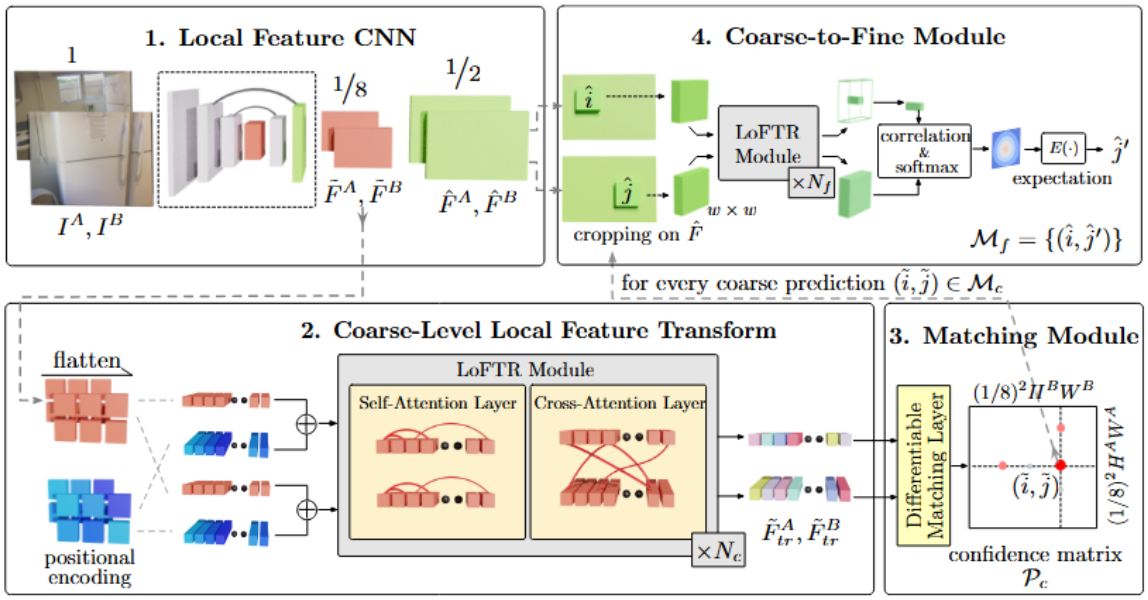


Figure 5: Visual overview of LoTFR

5.3.1 Local Feature Extraction

For a given pair of images, I_A and I_B, we use a *convolutional neural network* (CNN) network to produce:

- Fine-level features maps, F_F^A and F_F^B, where a window of 1/8 the size of the original dimension is run through to downsample the images into fine-level features.
- Coarse-level features map, F_C^A and F_C^B, where a window of 1/2 the size of the original dimension is run through to downsample the images into general-level features.

This is different from feature detection because we are extracting features across the entire image, as opposed to picking certain discernible features in the image. This has the advantage of providing a much larger space of features to compare between the images.

5.3.2 Local Feature Transformer (LoFTR) Module

With the coarse feature maps F_C^A and F_C^B , they are flattened and encoded with position information, and then passed on to the LoFTR module which consists of *attention layers*.

Attention layers, at its simplest, produce weighted representations of a sequence of components based on how relevant each component is to each other and to the sequence as a whole. It can be thought of as producing context-aware, feature representations that contain information relevant to all features.

The LoFTR module contains two kinds of attention layers, interlaced with each other:

- *Self-attention layers*: Applied to features within only one of F_C^A and F_C^B, this allows features within the same image to exchange contextual information, resolving ambiguities within the image.
- *Cross-attention layers*: Applied to features between F_C^A and F_C^B, this allows features to be connected across both images, strengthening global awareness of features from different perspectives of the same scene.

The LoFTR module then outputs transformed versions of the feature maps, tr^A and tr^B.

5.3.3 Matching Module

Given the transformed maps, F_{tr}^A and F_{tr}^B, we calculate the score matrix $\mathcal{S}$ with entries:

$$S_{ij} = \left\langle tr_i^A, tr_j^B \right\rangle$$

where $\langle\ \rangle$ denotes the inner product. We can then calculate the matching probability matrix $\mathcal{P}$ as:

$$\mathcal{P}_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})} \cdot \frac{\exp(S_{ij})}{\sum_n \exp(S_{nj})}$$

The above operation is also known as *dual softmax*. Note that i and j corresponds to flattened indices in the original features images I_A and I_B, and so $\mathcal{P}$ contain pointwise similarity between the points of each image. Referring i to I_A and j to I_B , we have the *mutual nearest neighbor* (MNN) of a given point i among all points j as

$$j^*(i) = \arg\max_j \mathcal{P}_{ij}$$

and the best match of j among all points i as:

$$i^*(j) = \arg\max_i \mathcal{P}_{ij}$$

With a given threshold θ, we then output the coarse-level match predictions as:

$$\mathcal{M} = \{(i,j)\ |(i,j) \in \mathrm{MNN}(\mathcal{P}),\ \mathcal{P}_{ij} \geq \theta\}$$

5.3.4 Coarse-to-Fine Module

With our coarse matches $\mathcal{M}$, containing pairs of flattened coordinates (i,j), we then find the corresponding coordinates in the fine feature maps F_F^A and F_F^B, denoted as $\hat{i}$ and $\hat{j}$ respectively.

We then take two sets of cropped windows of size $w \times w$ around $\hat{i}$ and $\hat{j}$, to produce windows W_A and W_B. Using a smaller version of the LoFTR module discussed in the preceding sections, we obtained transformed windows W_A^{tr} and W_B^{tr}.

The centre of W_A^{tr} is correlated with each point in W_B^{tr}, producing a heatmap that corresponds to the probability of points in the neighbourhood of $\hat{j}$ being matched with $\hat{i}$. Taking the expectation across all probable matches yields the point $\tilde{j}$, which is the predicted position of the point $\hat{i}$ in I_A in the image I_B.

The set of all fine-level matches $(\hat{i}, \tilde{j})$ is the final output of the algorithm.

5.3.5 Performance

The biggest advantage of the LoFTR algorithm is its ability to produce more feature matches compared to contemporary methods, making it easier to triangulate changes in position and rotation between images.

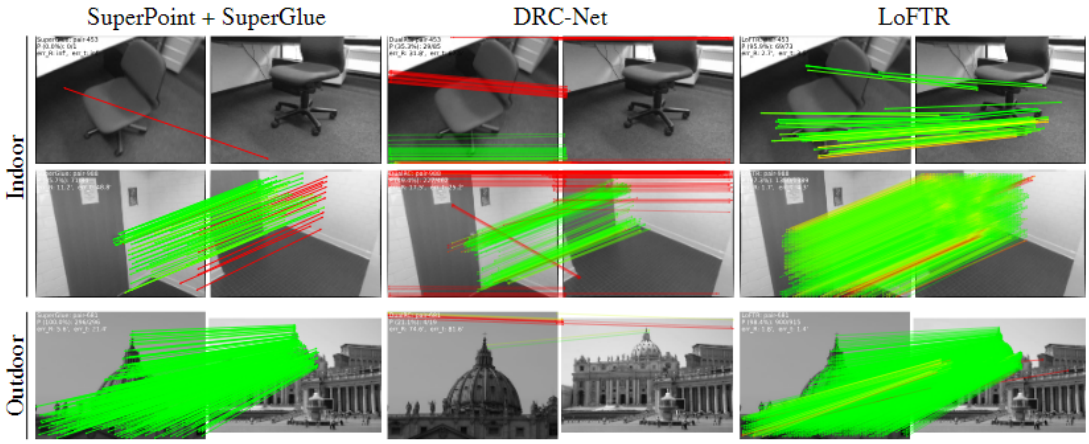


Figure 6: Comparison of feature matching with other algorithms

Category	Method	Homography est. AUC			#matches
		@3px	@5px	@10px	
Detector-based	D2Net [11]+NN	23.2	35.9	53.6	0.2K
	R2D2 [32]+NN	50.6	63.9	76.8	0.5K
	DISK [47]+NN	52.3	64.9	78.9	1.1K
	SP [9]+SuperGlue [37]	53.9	68.3	81.7	0.6K
Detector-free	Sparse-NCNet [33]	48.9	54.2	67.1	1.0K
	DRC-Net [19]	50.6	56.2	68.3	1.0K
	LoFTR-DS	**65.9**	**75.6**	**84.6**	1.0K

Figure 7: Benchmark on the HPatches dataset

This enables LoFTR to correctly predict changes in viewpoint, scoring a higher AUC score across different pixel-threshold values.

Category	Method	Pose estimation AUC		
		@5°	@10°	@20°
Detector-based	ORB [35]+GMS [2]	5.21	13.65	25.36
	D2-Net [11]+NN	5.25	14.53	27.96
	ContextDesc [27]+Ratio Test [26]	6.64	15.01	25.75
	SP [9]+NN	9.43	21.53	36.40
	SP [9]+PointCN [52]	11.40	25.47	41.41
	SP [9]+OANet [53]	11.76	26.90	43.85
	SP [9]+SuperGlue [37]	16.16	33.81	51.84
Detector-free	DRC-Net † [19]	7.69	17.93	30.49
	LoFTR-OT†	16.88	33.62	50.62
	LoFTR-OT	21.51	40.39	**57.96**
	LoFTR-DS	**22.06**	**40.8**	57.62

Figure 8: Benchmark on indoor pose estimation

6 Depth Estimation and Triangulation

Once a Structure from Motion (SfM) algorithm has estimated the camera parameters and derived feature matches, the next step is to use this information to estimate a depth for each of these keypoints.

6.1 Fundamental and Essential Matrices

This first step in estimating feature depths is to compute the fundamental matrix, F, and essential matrix, E . Based on a minimum of 8 feature matches, one can calculate the fundamental via the system of equations

$$x_i F x_j = 0$$

where x_i and x_j are corresponding feature points from two images [20]. This system of equations is normally solved with Singular Value decomposition. The essential matrix can then be computed with the camera's "intrinsic" matrix (made up of the camera's intrinsic parameters: focal length, distortion, etc.) K from the equation

$$E = K^T F K$$

6.2 Relative Rotation and Translation Matrix

Again, by singular value decomposition, one can then decompose the essential matrix to get a relative rotation matrix R and translation t between the two images ($E = [t]_\times R$) [21]. Because the essential matrix cannot determine whether the scene is in front or behind the cameras, this give four mathematically valid solutions.

This must be solved by the cheirality condition, which ensures that all points lie in front of the cameras. After evaluating all four possible depth estimates/projections based on the solutions, the solution with the most valid points (in front of the cameras) and the associated 3D projections are taken as correct.

6.3 Linear Triangulation

However, to use the cheirality condition one needs 3D projections or depth estimates, which must be calculated via a process of triangulation.

To linearly triangulate a point between two images, consider the projection matrices P_1 and P_2, projected 2D points x_1 and x_2, and triangulated 3D point X. One can then use the system of linear equations

$$\begin{bmatrix} x_1 \\ 1 \end{bmatrix} \times P_1 \begin{bmatrix} X \\ 1 \end{bmatrix} = 0$$
$$\begin{bmatrix} x_2 \\ 1 \end{bmatrix} \times P_2 \begin{bmatrix} X \\ 1 \end{bmatrix} = 0$$

to solve for the 3D triangulation X.

One can also refine these linearly triangulated points (which minimize the algebraic error) by applying a non-linear triangulation algorithm (which minimizes the reprojection error). The reprojection error can be computed as the error between the initial 2D measurements from the source images and the reprojected 3D points or, mathematically,

$$\min_X \sum_{j=\{1,2\}} (x_1^j - \frac{P_1^{j\mathrm{T}} \tilde{X}}{P_3^{j\mathrm{T}} \tilde{X}})^2 + (x_2^j - \frac{P_2^{j\mathrm{T}} \tilde{X}}{P_3^{j\mathrm{T}} \tilde{X}})^2 \tag{4}$$

where $\tilde{X}$ is the homogeneous representation of X [22]. It should be noted that to solve this non-linear optimization, one requires the initial linear triangulation as an initial guess.

Algorithms such as the aforementioned batch adjustment (or scaled batch adjustment), can also be used to refine the estimated points.

7 Dense Reconstruction

Once both camera parameters (from an SfM pipeline) and a sparse point cloud (from depth-estimation and triangulation) are known, a more complete 3D point cloud can be created through a process of dense reconstruction. In general, for each image used in the reconstruction process, it creates a dense depth map. These depth maps are then combined to form a dense reconstruction that can fully represent surface geometry [23]. In short, while feature mapping, detection, and triangulation were used to compute a relative depth map for a few points over entire 3D structure, dense reconstruction uses the derived camera positions to bridge the gap between a cloud of isolated point pairs to a near-continuous representation of the scene.

7.1 Dense Reconstruction Pipeline

Even with all camera parameters known and given a sparse point cloud, the process of dense reconstruction is complex and is comprised of multiple individual steps:

1. **Individual Image Depth Map Estimation:**

   First, a multi-view stereo (MVS) method is used to create a map that estimates the depth (and optionally the normal) for each pixel in each image. There are many algorithms and methods that can do this — as explained in the literature review of "Multi-View Stereo Revisited" [6] — including graph cuts, level set evolution, and mesh evolution. As noted by Goesele et al., more simple window-based algorithms can produce results with similar accuracy, though they often require more input views to create a complete reconstruction.

2. **Fusion of Depth Maps:**

   Once depth maps have been created/estimated for each image, these maps are amalgamated into a dense 3D point cloud. This step may still include filters for geometric and photometric consistency, in addition to the removal of outliers. It should be noted that the dense point cloud is independent from the sparse point cloud. That is, the step recalculates the depth and geometry of every pixel, including those that were used as keypoints.

   In general, this is done by first converting each pixel to a 3D point via some projection equation [24]. If one has, for a specific image and pixel (u, v), a rotation matrix R and a translation matrix t, we can reconstruct the 3D point X from the equation

$$X = R^{-1}(dK^{-1}[u, v, 1]^T - t) \tag{5}$$

   where d is the estimated depth from the depth map [24] [25]. Then, for each point, one can reproject and compare it to another images depth map using the inverse operation. Keeping points that are consistent across several views, the dense cloud can then be created by combining all remaining 3D points.

7.2 Primary MVS Method Families

7.2.1 PatchMatch Stereo

This method is the most commonly used and is a major component of the COLMAP, OpenMVS, and other dense reconstruction pipelines. It initializes by randomly assigning each pixel in an image a depth and normal, then recursively refining these depths and normals by both propagation and random search [26]. This algorithm/process is fast, GPU-friendly, and scales well to large datasets, yet retains an ability to process highly-detailed and textured scenes with accuracy [27].

7.2.2 Plane Sweeping

An older, more simple approach to MVS is plane sweeping. This method "sweeps" hypothetical planes at different depths, reprojecting images and evaluating for photometric/geometric consistency between them. For each pixel, the plane with the closest match is chosen as the pixel's depth. As may be apparent based on its methodology of evaluating each pixel for a number of planes, this approach can be extremely computationally costly and often results in lower-resolution point maps than a PatchMatch Stereo method.

7.2.3 Learning-Based MVS

This is a much more modern approach which employs deep learning and neural networks to learn to match 2D patterns with depth. These methods often move from course depth prediction to more fine-tuned estimates and can be extremely robust when lighting, texture, or structure are not easily identifiable. However, given its complexity and need for the neural network of deep learning model to be trained, these approaches can be slow and computationally intensive, thus making them less appealing for most MVS pipelines.

8 Surface Reconstruction, Meshing, and Visualization from Point Clouds

The last step in 3D reconstruction is to transform these raw points into a continuous 3D surface, add textures, and view the finished model after depth information or dense point clouds have been recovered. The procedure creates continuous surface representations (meshes) that are appropriate for rendering, analysis, and visualization from unstructured 3D point data.

8.1 Poisson Surface Reconstruction

We carry out a number of crucial preprocessing steps:

- **Voxel Grid Downsampling:**

  We use voxel-based downsampling, which preserves the overall shape while lowering point density, to guarantee computational tractability and uniform point distribution. The desired reconstruction resolution and the computational resources at hand are taken into consideration when determining the voxel size.

- **Normal Estimation:**

  For many reconstruction algorithms, especially implicit ones, consistent normal orientation is essential. Principal component analysis is used to compute normals on local neighborhoods, and local point density is used to automatically determine neighborhood size. We use a minimum spanning tree method to propagate consistent orientation throughout the point cloud for normal orientation.

- **Outlier Removal:**

  To get rid of false noise points, statistical outlier removal is used.

8.2 Surface Reconstruction Algorithms

8.2.1 Poisson Surface Reconstruction

It uses a set of oriented three-dimensional points to create a continuous, watertight surface. A point cloud is the first step in the process.

$$P = \{(p_i, n_i)\}_{i=1}^{N},$$

where $p_i \in \mathbb{R}^3$ are sample points on the surface and n_i are their corresponding oriented normals.

The gradient of this indicator function is associated with the surface's normal field. We assume in areas close to the surface.

$$\nabla\chi \approx V,$$

here the vector field V is created using the input normals. Consequently, the Poisson equation can be solved to recover the indicator function.

$$\nabla^2\chi = \nabla \cdot V.$$

The reconstructed surface is extracted as the zero-isosurface after χ has been solved.

$$S = \{x \in \mathbb{R}^3 : \chi(x) = 0\}.$$

The technique is noise-resistant and creates smooth, watertight surfaces. However, because normal orientation is crucial to the reconstruction, improperly oriented normals or inadequate preprocessing could result in artifacts. Because of its stability and mathematical elegance, Poisson surface reconstruction is still one of the most popular algorithms in photogrammetry, 3D scanning, and point-cloud processing [28].

8.3 Texturing the Mesh

The original input images can be used to texture the mesh after it has been created. Mapping color data from a 2D image back onto a 3D surface is called texturing. There are two primary methods.

8.3.1 Per-Vertex Color Mapping

By projecting each vertex of the 3D mesh into the calibrated input images, per-vertex texturing directly assigns a color to each vertex. The final color is typically calculated as an average or weighted average of the corresponding pixel values for each vertex, taking into account all images in which the point is visible. Because it only stores color at discrete mesh points and does not create texture atlases, this method is straightforward and computationally efficient. However, the resulting texture frequently looks smooth or blurry because color is interpolated across triangles, particularly when the mesh is coarse. Per-vertex coloring is helpful for working with lightweight or low-resolution meshes, debugging reconstruction quality, and rapid visualization despite its drawbacks [29, 30].

8.3.2 UV Texture Mapping

It generates a 2D texture atlas that, regardless of the mesh geometry, contains high-resolution color information. The process starts by unwrapping the mesh surface into a flat coordinate domain (the UV plane) and allocating a corresponding location in the texture image to each vertex or face. After each triangle is projected back into the input photos, the texture map is filled with the best-quality source image, which is selected based on visibility, angle, and resolution. This technique can maintain subtle visual elements like patterns, labels, and tiny surface variations that per-vertex coloring cannot because texture resolution is not dependent on mesh density. Because UV mapping provides the most visually accurate reconstruction of the original scene, it is the standard technique used in photogrammetry pipelines like COLMAP, Meshroom, and commercial 3D scanning software [30–32].

8.4 Visualization Tools

Point clouds, meshes, and textured 3D reconstructions can be visualized, examined, and improved using a range of software tools.

8.4.1 COLMAP

With its Structure-from-Motion and Multi-View Stereo pipelines, it offers an integrated viewer. Users can look at reconstructed meshes, sparse point clouds, dense point clouds, and camera poses. COLMAP's viewer is useful for diagnosing calibration errors, feature-matching failures, or poorly reconstructed regions because it shows the entire pipeline stages [31].

8.4.2 Meshroom (AliceVision)

Node-based workflows with automatic 3D reconstruction are its main focus. Its viewer focuses on interactive debugging of intermediate outputs, including feature correspondences and depth maps, as well as textured mesh inspection [32].

8.4.3 MeshLab

It's appropriate for full end-to-end photogrammetry projects due to its integration of texturing, mesh cleaning, and model export options [33].

8.4.4 Open3D

This Python/C++ library is intended for programmatic 3D data processing and visualization. It easily integrates with machine learning workflows and supports rendering point clouds, camera trajectories, depth images, and triangle meshes. Open3D is perfect for analytical examination of algorithmic outputs because of its interactive viewer's adjustable lighting, shading, and mesh-picking features [34].

9 Code Demonstration

We will demonstrate an example of 3D reconstruction based on a dataset of images of an Ammonite fossil taken from different angles using the OpenCV Python library [35].

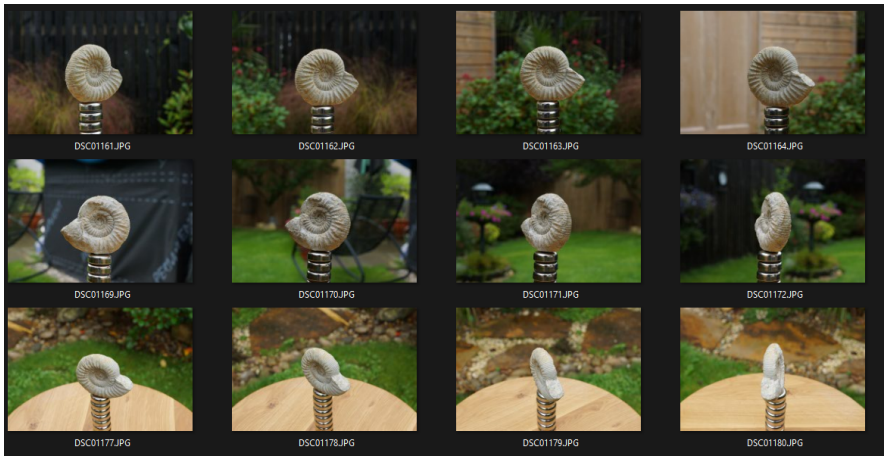


Figure 9: Pictures from the Ammonite dataset

9.1 Camera Parameter Estimation

The metadata in the images indicate that the pictures were taken with a Sony Alpha 77, with a sensor size of 23.5mm by 15.6mm and taken at a focal length of 50mm.

The pictures were originally of dimensions 6000 x 4000 pixels. To improve the speed of the reconstruction algorithm, we decided to compress the images to 1500 x 1000 pixels instead. This enables us to construct the intrinsic matrix as such:

We can now calculate the extrinsic matrix as such:

```
# Basic parameters
foc = 50
img_width = 1000
img_height = 1500
sens_width = 23.5
sens_length = 15.6

# Intrinsic parameters
fx = (foc / sens_width) * img_width
fy = (foc / sens_length) * img_width
cx = img_width / 2
cy = img_height / 2

# Construct intrinsic matrix
intrin = np.array([[fx, 0, cx],
                   [0, fy, cy],
                   [0,  0,  1]])

print(intrin)
```

✓ 0.0s

```
[[2.12765957e+03 0.00000000e+00 5.00000000e+02]
 [0.00000000e+00 3.20512821e+03 7.50000000e+02]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
```

Figure 10: Calculation of intrinsic matrix

9.2 Feature Detection and Matching

We used the SIFT algorithm to pick out features or *keypoints* from each image, because of its simplicity and effectiveness with large datasets. Matching keypoints between images this way requires us to do it with one pair of images at a time.

Observing the images in the Ammonite dataset in sequential order, we can see the camera rotating around the object at a given height, before switching to a different height repeating the process. To avoid matching across $102 \times 101 = 10302$ pairs, we can take advantage of the image order in the dataset and pair up nearby images to each other.

We decided to form sequential matching pairs consisting of the nth and $(n+1)^{th}$ images and skip matching pairs consisting of nth and $(n+2)^{th}$ pairs. We used the *FLANN* algorithm for feature matching by comparing keypoints in the image pair, and applying a ratio test to pick out feature matches that are distinct and stand out from the rest.

Feature matching

Our feature matching strategy is as follows:

- The images, when looked at sequentially, trace out a circular path around the object at different heights. Therefore, the order of the images follow a chronology.
- Instead pairing up all (102 x 101) pair of images and feature matching on them, we can feature match across successive pairs of images to give

```python
# This function uses FLANN + Lowe ratio test to feature match
def match_pair(img1, img2, ratio=0.75):
    # FLANN parameters for SIFT
    index_params = {'algorithm': 1, 'trees': 5}
    search_params = {'algorithm': 1, 'trees': 5}
    flann = cv2.FlannBasedMatcher(index_params, search_params)

    # Create and compare matches, by finding 2 nearest neighbors of each match,
    # and then applying the ratio test
    matches = flann.knnMatch(img1.des, img2.des, k = 2)
    good_matches = []

    for m,n in matches:
        if m.distance < ratio * n.distance:
            good_matches.append(m)


    return good_matches
```

✓ 0.0s

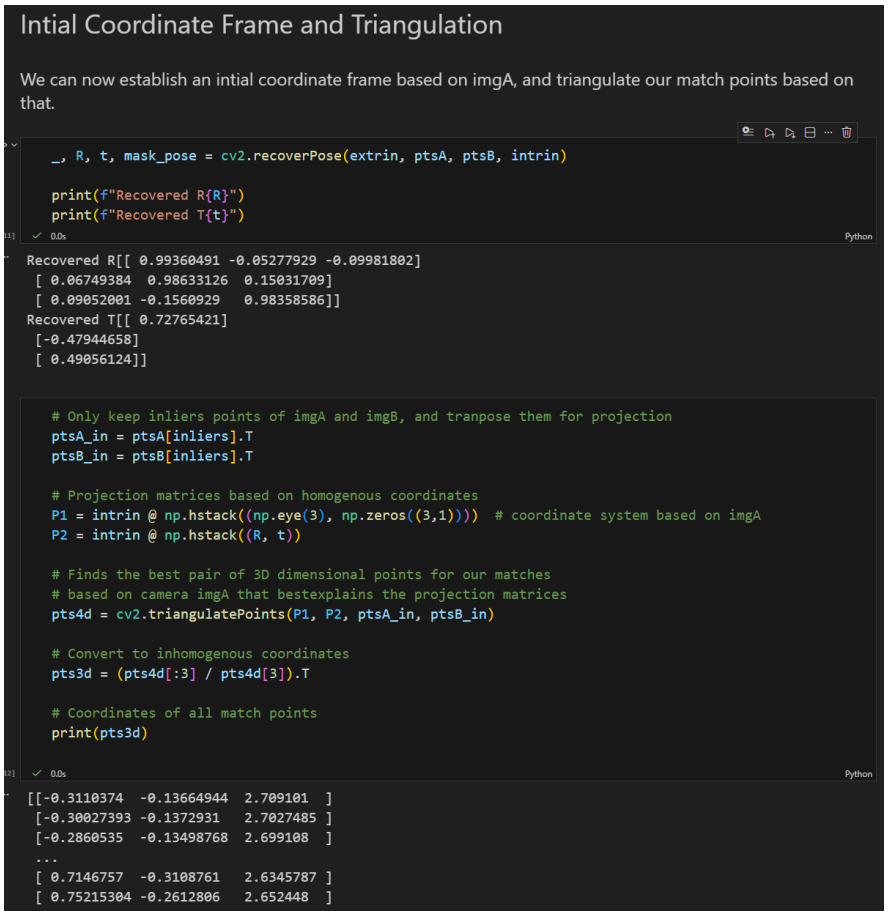
```python
#   We will store our matches in a dict, with the key corresponding
#   to the indices of the image pair, and the value containing
#   the matched descriptions

match_graph = {}

for i in tqdm(range(len(frames) - 1), desc="Sequential Matching"):
    match_graph[(i, i+1)] = match_pair(frames[i], frames[i+1])

#   We will also do skip matches, where the i-th item is matched with the i+2-th item
#   for robustness

for i in tqdm(range(len(frames) - 2), desc="Skip Matching"):
    match_graph[(i, i+2)] = match_pair(frames[i], frames[i+2])
```

Figure 11: Enter Caption

9.3 Setting up the Coordinate Frame

To set up a coordinate system where we can triangulate and place our points, we need to pick one of the images as a reference perspective and map all of the image relative to this perspective. We picked the image pair with the highest number of feature matches, and set one image as the reference frame.

We then find the rotation and translation matrix from the reference image to the other image of the pair using a pose estimation algorithm. This enables us to generate a set of 3D points that best explain how the matched features move between the image pair given our rotation and translation matrix.



Intial Coordinate Frame and Triangulation

We can now establish an intial coordinate frame based on imgA, and triangulate our match points based on that.

```python
_, R, t, mask_pose = cv2.recoverPose(extrin, ptsA, ptsB, intrin)

print(f"Recovered R{R}")
print(f"Recovered T{t}")
```

[11] ✓ 0.0s Python

```
Recovered R[[ 0.99360491 -0.05277929 -0.09981802]
 [ 0.06749384  0.98633126  0.15031709]
 [ 0.09052001 -0.1560929   0.98358586]]
Recovered T[[ 0.72765421]
 [-0.47944658]
 [ 0.49056124]]
```

```python
# Only keep inliers points of imgA and imgB, and tranpose them for projection
ptsA_in = ptsA[inliers].T
ptsB_in = ptsB[inliers].T

# Projection matrices based on homogenous coordinates
P1 = intrin @ np.hstack((np.eye(3), np.zeros((3,1))))  # coordinate system based on imgA
P2 = intrin @ np.hstack((R, t))

# Finds the best pair of 3D dimensional points for our matches
# based on camera imgA that bestexplains the projection matrices
pts4d = cv2.triangulatePoints(P1, P2, ptsA_in, ptsB_in)

# Convert to inhomogenous coordinates
pts3d = (pts4d[:3] / pts4d[3]).T

# Coordinates of all match points
print(pts3d)
```

[12] ✓ 0.0s Python

```
[[-0.3110374  -0.13664944  2.709101  ]
 [-0.30027393 -0.1372931   2.7027485 ]
 [-0.2860535  -0.13498768  2.699108  ]
 ...
 [ 0.7146757  -0.3108761   2.6345787 ]
 [ 0.75215304 -0.2612806   2.652448  ]
```

Figure 12: Triangulation of our initial set of feature matches

9.4 Structure From Motion

As a setup, we create a mapping structure that keeps track of:

- XYZ-coordinates of each feature match and which images observe it.
- The pose in each image, in terms of rotation and translation matrices relative to our reference image.
- Which images have been registered, which only consists of our initial image pair so far.

We set up a loop where as long as the number of registered images is less than the total number of images, we do:

9.4.1 Find Next Best Image

We go through all image pairs where one image is registered and the other one is not. We use a Point-N-Perspective (PnP) algorithm where we compare keypoints in the registered image which has XYZ-coordinates with those in the unregistered one, allowing us to correlated 2D points in the unregistered image to 3D coordinates. We pick the candidate image pair to be registered based on the number of such 2D to 3D correspondences.

Once, we have our next best image, we find keypoints in the registered image, and compare it to the keypoints in the best next image. This allow us to correspond 3D points in the map_points, with the 2D points in each image.

```python
def get_pnp_match(new_k, registered, map_points, frames, match_graph):
    pts3d = []
    pts2d = []

    # Explore all matches involving new_k
    for (i, j), matches in match_graph.items():
        if i in registered and j == new_k:
            for m in matches:
                # Retrieve image coords in both images
                kp_i = m.queryIdx
                kp_k = m.trainIdx

                # search through map_points for a point observed in image i
                for P in map_points:
                    if i in P['obs'] and P['obs'][i] == kp_i:
                        pts3d.append(P['coord'])
                        pts2d.append(frames[new_k].kp[kp_k].pt)

        if j in registered and i == new_k:
            for m in matches:
                kp_j = m.trainIdx
                kp_k = m.queryIdx

                for P in map_points:
                    if j in P['obs'] and P['obs'][j] == kp_j:
                        pts3d.append(P['coord'])
                        pts2d.append(frames[new_k].kp[kp_k].pt)

    return np.array(pts3d, dtype=np.float32), np.array(pts2d, dtype=np.float32)
```

✓ 0.0s Python

Figure 13: The PnP algorithm used

9.4.2 Pose estimation

With the 2D-3D correspondences from the previous section, we use pose estimation to get the rotation and translation matrices for this new image based on the registered one.

```python
# ---- 3. Solve camera pose via PnP ----
success, rvec, tvec = cv2.solvePnP(pts3d, pts2d, intrin, None, flags=cv2.SOLVEPNP_ITERATIVE)
if not success:
    print(f"PnP failed for view {new_view}. Skipping.")
    registered.add(new_view)
    poses[new_view] = poses[reg_view]
    continue

R, _ = cv2.Rodrigues(rvec)
t = tvec.reshape(3, 1)
poses[new_view] = (R, t)
registered.add(new_view)
```

Figure 14: Code for pose estimation

9.4.3 Point Triangulation

Using our projection matrices and our 2D-3D correspondences,, we can then get XYZ-coordinates for all feature matches in the candidate image pair, by using a triangulation algorithm.

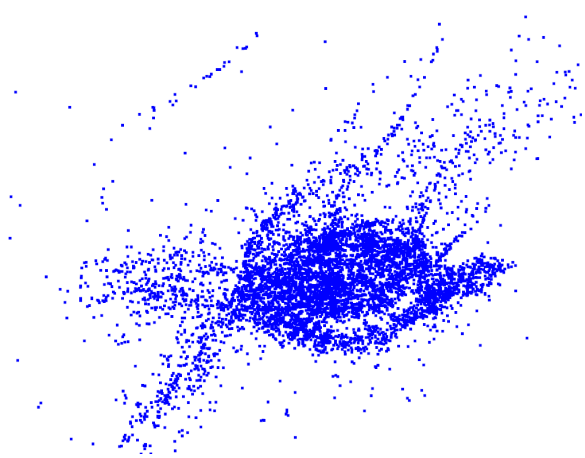
```python
ptsA, ptsB = [], []
for m in matches:
    if not swapped:
        ptsA.append(frames[reg_view].kp[m.queryIdx].pt)
        ptsB.append(frames[new_view].kp[m.trainIdx].pt)
    else:
        ptsA.append(frames[reg_view].kp[m.trainIdx].pt)
        ptsB.append(frames[new_view].kp[m.queryIdx].pt)

ptsA = np.float32(ptsA).T
ptsB = np.float32(ptsB).T

P1 = intrin @ np.hstack([poses[reg_view][0], poses[reg_view][1]])
P2 = intrin @ np.hstack([poses[new_view][0], poses[new_view][1]])

pts4d = cv2.triangulatePoints(P1, P2, ptsA, ptsB)
new_pts3d = (pts4d[:3] / pts4d[3]).T

# ---- 5. Filter points: behind camera / extreme depth ----
mask = (new_pts3d[:, 2] > 0) & (new_pts3d[:, 2] < 10)
new_pts3d = new_pts3d[mask]
matches = np.array(matches)[mask]
```

Figure 15: Code for point triangulation, with some filtering to get rid of points of extreme depth

We then add these points to our mapping structured, provided that there are no nearby points. We then repeat the process by registering in the candidate image, and pick a new candidate image pair.

9.5 Output

Using our XYZ-points, we used the Open3D library to visualize the points as such:



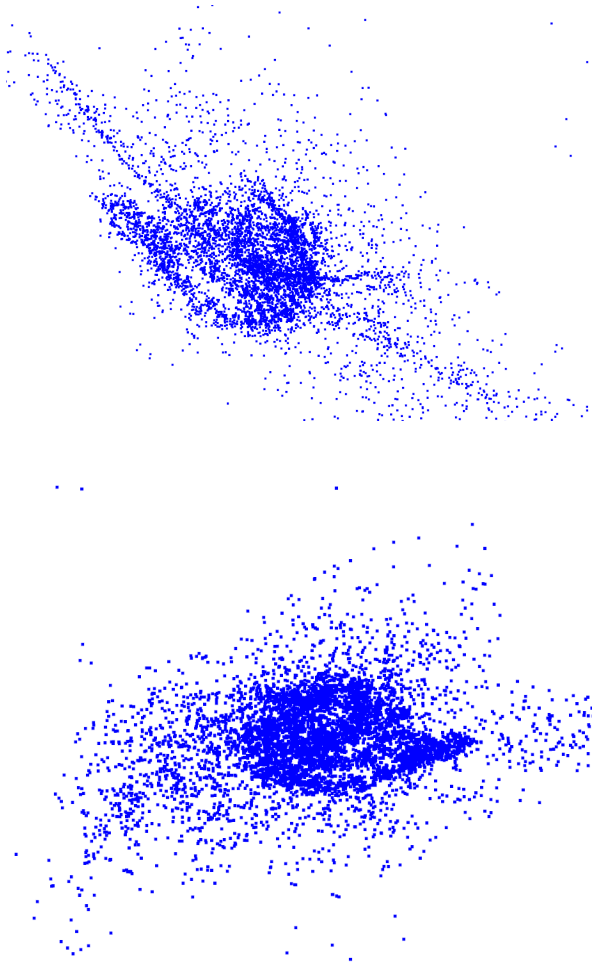


Figure 16: The sparse point reconstruction at different angles

We can see that Ammonite fossil has been picked up alongside with some errenous background features. A better reconstruction could be obtained through:

- Changes to various tolerance parameters used throughout the algorithm.
- Manual pruning of background of points.
- Use a more robust method of feature matching that eliminates background points.
- Use more image pairs as the Ammonite fossil is the only common feature, this would remove background points.
- Isolating the object better in the initial images.

10 Conclusion and Findings

Given all the different approaches and methods that we have shown for each stage of the reconstruction process, there is no singular method that works best in all situations. The field of 3D reconstruction is a changing one, with methods such as LoFTR coming out as neural network and transformer architectures continue to develop, and improvements in computational power enabling previously infeasible

algorithms to be used in day-to-day cases.

This dependency on context for determining a reconstruction approach is highlighted well by the paper "Plan3D: Viewpoint and Trajectory Optimization for Aerial Multi-View Stereo Reconstruction" [10]. Though SfM and MVS pipelines are becoming much more efficient and accurate, the process of applying a 3D reconstruction technique relies as much on the non-technical constraints as much as the technical. That is, understanding how each reconstruction tool can be used within a specific context and how that context can be changed to better align with tools is far more important than any single advancement in the general field. Usually, this means that better or more varied imagery will produce better results and optimizing this imagery for preprocessing later can be vital to dense, accurate reconstructions.

References

[1] R. Szeliski. [Online]. Available: https://library.huree.edu.mn/data/202295/2024-06-03/ComputerVision-AlgorithmsandApplications2ndEdition,RichardSzeliski.pdf

[2] R. Hartley and A. Zisserman, "Multiple view geometry in computer vision." [Online]. Available: https://www.cambridge.org/core/books/multiple-view-geometry-in-computer-vision/0B6F289C78B2B23F596CAA76D3D43F7A

[3] F. Qureshi. [Online]. Available: https://csundergrad.science.uoit.ca/courses/cv-notes/notebooks/01-image-formation.html

[4] A. Torralba, P. Isola, and W. Freeman, "Foundations of computer vision." [Online]. Available: https://visionbook.mit.edu/imaging_geometry.html

[5] H. Han and H. Yang, "Building Rome with Convex Optimization," *School of Engineering and Applied Sciences, Harvard University*, 6 2025. [Online]. Available: https://doi.org/10.15607/rss.2025.xxi.032

[6] M. Goesele, B. Curless, and S. Seitz, "Multi-View Stereo Revisited," *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2, pp. 2402–2409, 7 2006. [Online]. Available: https://doi.org/10.1109/cvpr.2006.199

[7] D. M. Greig, B. T. Porteous, and A. H. Seheult, "Exact maximum a posteriori estimation for binary images," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 51, no. 2, pp. 271–279, 12 2018. [Online]. Available: https://doi.org/10.1111/j.2517-6161.1989.tb01764.x

[8] Z. Wang, D. Acuna, H. Ling, A. Kar, and S. Fidler, "Object Instance Annotation with Deep Extreme Level Set Evolution," 2019. [Online]. Available: https://www.cs.toronto.edu/~zianwang/DELSE/delse.html

[9] A. Zaharescu, E. Boyer, and R. Horaud, "Topology-Adaptive Mesh Deformation for Surface Evolution, Morphing, and Multi-View Reconstruction," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 4, 12 2020.

[10] B. Hepp, M. Niebner, and O. Hilliges, "Plan3D: Viewpoint and Trajectory Optimization for Aerial Multi-View Stereo Reconstruction," *ACM Transactions on Graphics*, vol. 38, no. 1, 12 2018. [Online]. Available: https://dl-acm-org.proxy.lib.uwaterloo.ca/doi/pdf/10.1145/3233794

[11] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, *NERF: Representing scenes as neural radiance fields for view synthesis*, 1 2020. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_24

[12] H. Longuet-Higgins, "A computer algorithm for reconstructing a scene from two projections," *Readings in Computer Vision*, p. 61–62, 1987.

[13] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.

[14] OpenCV, "Introduction to SIFT (Scale-Invariant Feature Transform)." [Online]. Available: https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

[15] L. Sigal, "Lecture Notes, Week 8: SIFT," University of British Columbia, Tech. Rep., 2022. [Online]. Available: https://www.cs.ubc.ca/~lsigal/425_2022W2/NotesW8.pdf

[16] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *2011 International Conference on Computer Vision*, 2011.

[17] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *European Conference on Computer Vision*, 2010. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-15561-1_56

[18] OpenCV, "Orb — opencv documentation," 2023. [Online]. Available: https://docs.opencv.org/4.x/d1/d89/tutorial_py_orb.html

[19] J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, "Loftr: Detector-free local feature matching with transformers," 2021. [Online]. Available: https://arxiv.org/abs/2104.00680

[20] F. Seliya, "Building Built in Minutes: Structure from motion," 3 2025. [Online]. Available: https://sites.google.com/view/farhanseliyarobotics/projects/classical-computer-vision/structure-from-motion

[21] K. Matzen, "Structure from Motion," Cornell University, Tech. Rep., 4 2016. [Online]. Available: https://www.cs.cornell.edu/courses/cs4670/2016sp/lectures/lec31_sfm_web.pdf

[22] C. . I. Team, "Project 2: Structure from Motion," University of Pennsylvania, Tech. Rep., 4 2015. [Online]. Available: https://www.cis.upenn.edu/~cis580/Spring2015/Projects/proj2/proj2.pdf

[23] Y. Li, "3 Dimensional Dense Reconstruction: A Review of Algorithms and Dataset," *Electrical and Computer Engineering Technology Department*, 4 2023. [Online]. Available: https://arxiv.org/pdf/2304.09371v1

[24] M. Rao, "CV-17 (Depth Image to Point Cloud)," 2 2025, accessed on Feb. 2, 2025. [Online]. Available: https://medium.com/\%40monishatemp20/cv-17-depth-image-to-pointcloud-d83247a5d8e8

[25] Y. Xiang, "Camera Projection," University of Texas at Dallas, Tech. Rep., 2 2024. [Online]. Available: https://yuxng.github.io/Courses/CS4391Spring2024/lecture_11_camera_projection.pdf

[26] M. Bleyer, C. Rhemann, and C. Rother, "PatchMatch Stereo - Stereo Matching with Slanted Support Windows," *Microsoft Research*, pp. 14.1–14.11, 1 2011. [Online]. Available: https://doi.org/10.5244/c.25.14

[27] T. Gee, "PatchMatch Stereo matching," 1 2023. [Online]. Available: https://wildboar-dev.github.io/PatchMatch/

[28] M. Kazhdan and H. Hoppe, "Screened poisson surface reconstruction," *ACM Transactions on Graphics (TOG)*, vol. 32, no. 3, pp. 1–13, 2013. [Online]. Available: https://hhoppe.com/screenedpoisson.pdf

[29] M. Pollefeys *et al.*, "Visual modeling with a hand-held camera," *International Journal of Computer Vision*, 2004.

[30] M. Wächter, N. Moehrle, and M. Goesele, "Let there be color! large-scale texturing of 3d reconstructions," in *European Conference on Computer Vision*, 2014.

[31] J. L. Schönberger and J.-M. Frahm, "COLMAP: A general-purpose structure-from-motion pipeline," https://colmap.github.io, 2016.

[32] AliceVision, "Meshroom: Photogrammetry pipeline," https://alicevision.org, 2018.

[33] P. Cignoni *et al.*, "Meshlab: An open-source mesh processing tool," https://www.meshlab.net, 2008.

[34] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3d: A modern library for 3d data processing," *arXiv preprint arXiv:1801.09847*, 2018.

[35] K. O. A. Stéphane, "3d object reconstruction from images," Jun 2024. [Online]. Available: https://www.kaggle.com/datasets/kpoviesistphane/3d-object-reconstruction-from-images